

23CSE304 - EMBEDDED SYSTEMS

MINI PROJECT REPORT

on

FOOD SPOILAGE RISK INDICATOR

SUBMITTED BY:

Harshini R P - NC.SC.U4CSE24245

Pinnamareddy Ramya Sri - NC.SC.U4CSE24240

R S Vidhushaa - NC.SC.U4CSE24223

Karthiga R - NC.SC.U4CSE24211

Amirthaa - NC.SC.U4CSE24202

ABSTRACT

The Food Spoilage Risk Indicator Using Embedded System is a low-cost embedded monitoring system designed to observe environmental conditions associated with food-storage and spoilage risk. The system uses an ESP32 microcontroller as the main processing unit. A DHT11 sensor is used to measure temperature and humidity, while an MQ-135 sensor provides an analog gas/VOC response. The collected sensor readings are continuously processed by the ESP32 and compared with experimentally selected threshold values.

When the temperature, humidity, or gas-response value exceeds its corresponding threshold, the system activates a buzzer to provide an audible warning. When all measured values remain within the selected limits, the system displays a normal-storage message and keeps the buzzer OFF. The system is intended as an early-warning prototype. It does not directly confirm that food is spoiled. The thresholds are experimental and should be calibrated according to the food type, sensor placement, and storage environment. The project demonstrates how embedded systems and sensors can be combined to provide simple monitoring and alert functionality.

INTRODUCTION

Food quality can be affected by environmental conditions such as temperature, humidity, and gases or volatile compounds released during deterioration. Continuous monitoring of these conditions can help identify storage environments that require attention. Manual inspection of food-storage conditions does not provide continuous measurement. An embedded system can overcome this limitation by collecting sensor readings at regular intervals and comparing them with selected limits.

The proposed Food Spoilage Risk Indicator uses an ESP32 as the main controller. The DHT11 sensor measures temperature and humidity, while the MQ-135 sensor detects changes in gas/VOC response. The ESP32 processes these values and determines whether an alert condition exists. When an alert condition is detected, an audible buzzer is activated. The sensor readings and system status are also displayed through the Arduino Serial Monitor.

BACKGROUND

Food-storage environments need suitable environmental conditions to maintain food quality. Changes in temperature and humidity can affect storage conditions, while gases or volatile compounds may change during deterioration.

The project uses an embedded approach to monitor selected environmental parameters continuously.

The basic idea is:

Sensors → ESP32 → Data Processing → Threshold Comparison → Alert

The ESP32 receives data from the sensors and makes a simple threshold-based decision.

The system does not attempt to identify a specific food product or directly determine whether food is safe to consume. Instead, it provides an indication that measured conditions have moved beyond selected limits.

PROBLEM STATEMENT

Food-storage conditions may change without being continuously noticed. Manual monitoring may not provide repeated measurements of temperature, humidity, and gas-related environmental changes.

Therefore, there is a need for a compact and economical embedded system that can:

- Continuously monitor selected storage parameters.
- Measure temperature and humidity.
- Detect changes in gas/VOC response.
- Compare sensor readings with selected limits.
- Provide an immediate warning when an alert condition occurs.

PROPOSED SOLUTION

The proposed system uses an ESP32, DHT11 sensor, MQ-135 sensor, and buzzer to create a simple food-spoilage-risk monitoring prototype.

The system monitors the sensor values and activates the buzzer whenever at least one selected parameter exceeds its threshold.

OBJECTIVES

The main objectives of the project are:

1. To measure temperature using the DHT11 sensor.
2. To measure relative humidity using the DHT11 sensor.
3. To measure changes in gas/VOC response using the MQ-135 sensor.
4. To process sensor readings using an ESP32 microcontroller.
5. To compare sensor readings with selected threshold values.
6. To activate an audible buzzer when an alert condition occurs.
7. To display sensor values and system status through the Serial Monitor.
8. To develop a simple and economical embedded prototype.
9. To provide a system that can be expanded for future remote monitoring.

SCOPE

The scope of this project is limited to monitoring environmental parameters associated with food storage.

The prototype measures:

- Temperature
- Humidity
- Gas/VOC response

The ESP32 processes these values and compares them with predefined experimental thresholds.

The system provides:

- Sensor monitoring
- Threshold comparison
- Audible alert
- Serial Monitor output
- Normal/alert status indication

Scope for Future Development

The ESP32 provides possibilities for:

- Wireless monitoring
- IoT connectivity
- Data storage
- Mobile/web dashboards
- Additional sensors
- Food-specific calibration

The current prototype itself remains a local monitoring and alert system.

HARDWARE REQUIREMENTS

The hardware requirements are:

ESP32 Dev Module

Acts as the main controller.

DHT11 Sensor

Used to measure:

- Temperature
- Relative humidity

MQ-135 Sensor

Used to measure changes in gas/VOC response.

Buzzer

Provides an audible warning.

Breadboard

Used for temporary circuit assembly.

Jumper Wires

Used to make electrical connections.

USB Cable

Used for powering/programming the ESP32.

Computer

Used with Arduino IDE for programming and monitoring.

These are the hardware requirements specified in the original project.

SOFTWARE REQUIREMENTS

Arduino IDE

Arduino IDE is used to write, compile, and upload the program to the ESP32.

ESP32 Board Package

The ESP32 board package provides development support for programming the ESP32 through Arduino IDE.

DHT Sensor Library

The DHT library is used for reading temperature and humidity from the DHT11 sensor.

Embedded C/C++

The program is written using the Arduino C/C++ programming environment.

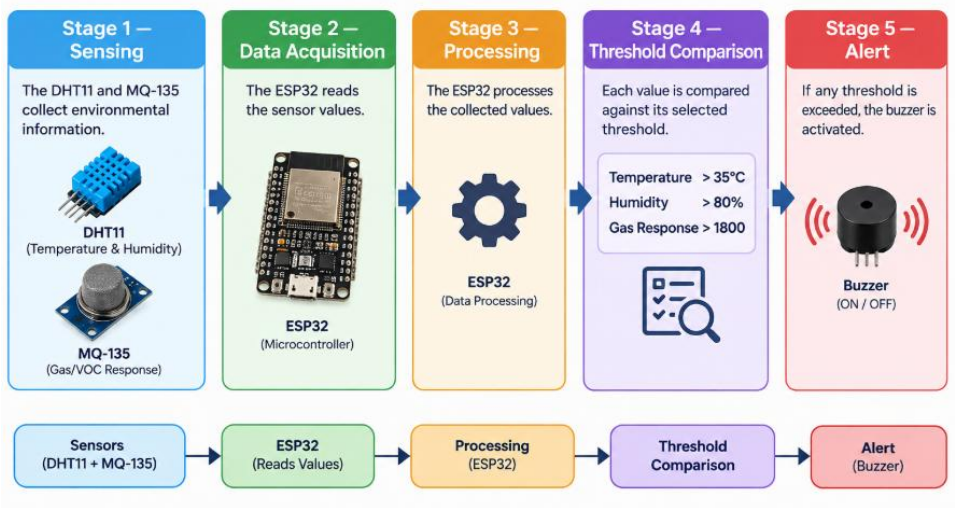
Serial Monitor

The Serial Monitor is used to display:

- Temperature
- Humidity
- MQ-135 gas response
- Warning status
- Normal storage status

The project uses a Serial Monitor speed of 115200 baud.

SYSTEM ARCHITECTURE



The complete system consists of five major stages:

Stage 1 - Sensing

The DHT11 and MQ-135 collect environmental information.

Stage 2 - Data Acquisition

The ESP32 reads the sensor values.

Stage 3 - Processing

The ESP32 processes the collected values.

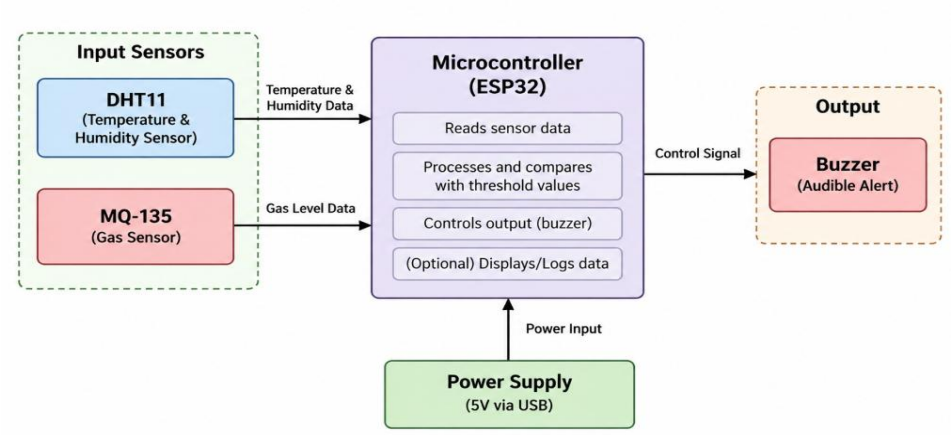
Stage 4 - Threshold Comparison

Each value is compared against its selected threshold.

Stage 5 - Alert

If any threshold is exceeded, the buzzer is activated.

BLOCK DIAGRAM



The block diagram represents the flow from sensor inputs to ESP32 processing, threshold comparison, and final alert decision.

WORKING PRINCIPLE

The system works in the following sequence:

1. The ESP32 initializes the DHT11 sensor and buzzer.
2. The DHT11 measures temperature and relative humidity.
3. The MQ-135 produces an analog gas/VOC response.
4. The ESP32 reads the sensor values.
5. The values are compared with predefined thresholds.
6. If any value exceeds its threshold, an alert condition is generated.
7. The buzzer is activated.
8. A warning message is displayed on the Serial Monitor.
9. If all values remain within their limits, the buzzer remains OFF.
10. The system waits for the next reading and repeats the process.

The original project specifies this repeated sensor-reading and threshold-comparison process.

COMPONENT DESCRIPTION

1. ESP32 Dev Module

The ESP32 is the central controller of the project.

Functions

- Reads sensor values.
- Processes sensor data.
- Performs threshold comparisons.
- Controls the buzzer.
- Sends information to the Serial Monitor.

Connections

The ESP32 uses:

- GPIO 4 for DHT11 data.
- GPIO 34 for MQ-135 analog output.
- GPIO 25 for buzzer control.

The ESP32 therefore acts as the communication and processing unit between the sensors and alert device.

2. DHT11 Temperature and Humidity Sensor

The DHT11 sensor is used to measure two environmental parameters:

Temperature

Temperature information is used to determine whether the measured environment has crossed the selected temperature limit.

Humidity

Relative humidity is monitored because excessive humidity can indicate an undesirable storage environment.

Connection

The DHT11 data pin is connected to:

DHT11 DATA → ESP32 GPIO 4

The sensor is powered using the specified 3.3 V connection and shares the common ground with the ESP32.

3. MQ-135 Gas/VOC Sensor

The MQ-135 is used to detect changes in gas/VOC response.

The sensor provides an analog output that is read by the ESP32.

Connection

MQ-135 AO → ESP32 GPIO 34

The project specifies VIN/5 V as the MQ-135 supply and common ground.

Important Safety Consideration

The MQ-135 analog output must remain within the ESP32 ADC input range. If the analog output can exceed 3.3 V, a suitable voltage divider should be used before connecting it to the ESP32 ADC input.

BUZZER

Component	Pin	ESP32 Connection	Notes
 DHT11 (Temperature & Humidity)	DATA	GPIO 4	Sensor data pin
	VCC	3.3 V	Sensor power supply
	GND	GND	Common ground
 MQ-135 (Gas/VOC Sensor)	AO	GPIO 34	Analog output (may need voltage divider if required)
	VCC	VIN / 5 V	Sensor power supply
	GND	GND	Common ground
 Buzzer	+	GPIO 25	Positive terminal
	–	GND	Negative terminal

ALGORITHM

Step 1

Start the system.

Step 2

Initialize the ESP32, DHT11 sensor and buzzer.

Step 3

Read temperature from the DHT11.

Step 4

Read humidity from the DHT11.

Step 5

Read the analog MQ-135 value.

Step 6

Check whether the DHT11 readings are valid.

Step 7

Compare temperature with 35°C.

Step 8

Compare humidity with 80%.

Step 9

Compare MQ-135 gas response with 1800.

Step 10

If any value exceeds its threshold, activate the buzzer.

Step 11

Display **“SPOILAGE RISK DETECTED”**.

Step 12

If all values are within limits, keep the buzzer OFF.

Step 13

Display **“NORMAL STORAGE CONDITIONS”**.

Step 14

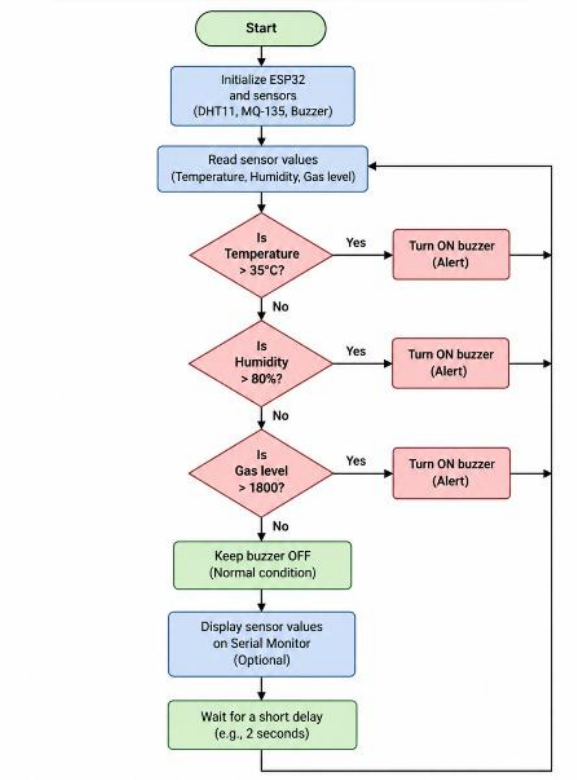
Wait for the next reading.

Step 15

Repeat continuously.

The threshold logic is specified in the project as temperature > 35°C OR humidity > 80% OR MQ-135 response > 1800.

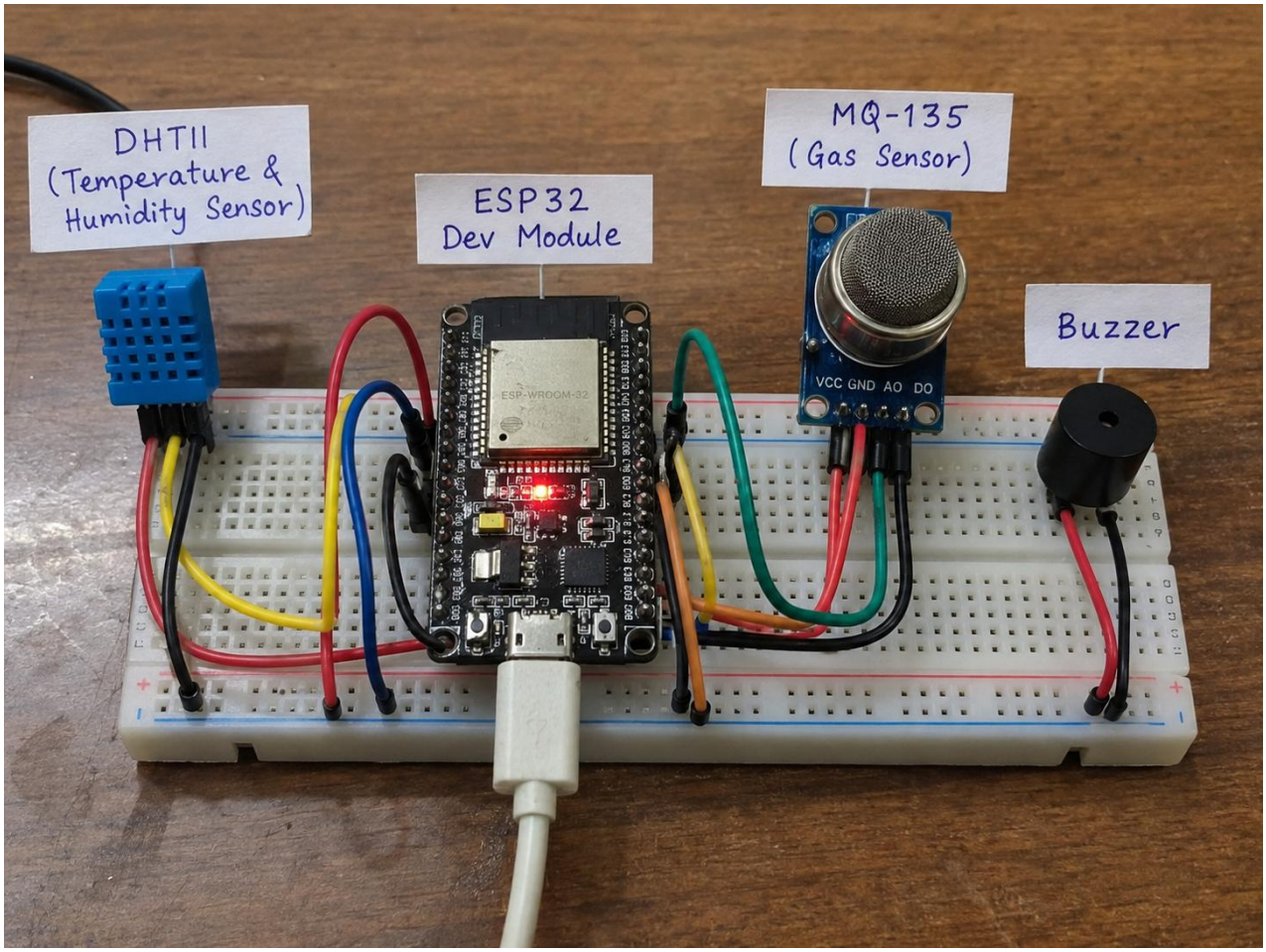
FLOWCHART



CIRCUIT / PROTOTYPE

The prototype consists of:

- ESP32 Dev Module
- DHT11 sensor
- MQ-135 gas/VOC sensor
- Buzzer
- Breadboard
- Jumper wires
- USB connection



PIN CONNECTIONS

Device	Pin	ESP32
DHT11	VCC	3.3 V
DHT11	DATA	GPIO 4
DHT11	GND	GND
MQ-135	VCC	VIN / 5 V
MQ-135	AO	GPIO 34 through suitable voltage divider if required
MQ-135	GND	GND
Buzzer	+	GPIO 25
Buzzer	–	GND

PROGRAM CODE

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
#define MQ135_PIN 34
#define BUZZER_PIN 25

float temperatureThreshold = 35.0;
float humidityThreshold = 80.0;
int gasThreshold = 1800;

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(MQ135_PIN, INPUT);
  ledcAttach(BUZZER_PIN, 3000, 8);
  ledcWriteTone(BUZZER_PIN, 0);
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  int gasValue = analogRead(MQ135_PIN);

  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("DHT11 Sensor Error!");
    delay(2000);
    return;
  }

  bool tempAlert = temperature > temperatureThreshold;
  bool humidityAlert = humidity > humidityThreshold;
  bool gasAlert = gasValue > gasThreshold;

  Serial.println("--------------------------------");
  Serial.print("Temperature: "); Serial.print(temperature); Serial.println(" °C");
  Serial.print("Humidity: "); Serial.print(humidity); Serial.println(" %");
  Serial.print("MQ-135 Gas Response: "); Serial.println(gasValue);

  if (tempAlert || humidityAlert || gasAlert) {
    Serial.println("SPOILAGE RISK DETECTED");
    ledcWriteTone(BUZZER_PIN, 3000);
  } else {
    Serial.println("NORMAL STORAGE CONDITIONS");
    ledcWriteTone(BUZZER_PIN, 0);
  }

  delay(3000);
}
```

IMPLEMENTATION

The prototype is assembled using a solderless breadboard.

The ESP32 is used as the central processing unit.

The DHT11 sensor is connected to GPIO 4.

The MQ-135 analog output is connected to GPIO 34.

The buzzer is connected to GPIO 25.

All modules share a common ground.

During operation, the ESP32 continuously reads the sensor values and compares them with the predefined thresholds.

The results are displayed on the Serial Monitor, and the buzzer provides an audible warning whenever an alert condition occurs.

RESULTS

The prototype is designed to produce the following responses:

Test Condition	Observed Response
Temperature within limit	No temperature warning; buzzer remains OFF if other values are normal.
Humidity within limit	No humidity warning; buzzer remains OFF if other values are normal.
Gas response above threshold	Gas warning displayed and buzzer turns ON.
Any one alert condition	Overall spoilage-risk warning is displayed and buzzer turns ON.
All values within limits	Normal storage condition is displayed and buzzer remains OFF.

Result Interpretation

If all measured values remain below their thresholds, the Serial Monitor displays:

NORMAL STORAGE CONDITIONS

If at least one value exceeds its threshold, the Serial Monitor displays:

SPOILAGE RISK DETECTED

and the buzzer is activated.

These response conditions are documented in the original project results.

APPLICATIONS

The system can be applied to:

1. Small food-storage containers.
2. Food-storage cabinets.
3. Educational embedded-system demonstrations.
4. Food-processing and storage demonstrations.
5. Experimental monitoring of storage environments.
6. Future IoT-based food-condition monitoring systems.

ADVANTAGES

The main advantages are:

- Low-cost prototype.
- Compact system.
- Simple sensor-based monitoring.
- Continuous measurement.
- Immediate audible warning.
- Simple threshold-based operation.
- Easy modification of threshold values.
- ESP32 provides scope for future wireless connectivity.

These applications and advantages are based on the original project description.

LIMITATIONS

The system has the following limitations:

1. General Gas Sensor

The MQ-135 is a general gas/VOC sensor and does not directly identify food spoilage.

2. Experimental Thresholds

The selected thresholds may vary according to food type, sensor placement, and environmental conditions.

3. DHT11 Accuracy

The DHT11 provides basic temperature and humidity measurement rather than high-precision measurement.

4. Local Alert

The current prototype uses a local buzzer and does not provide cloud monitoring.

5. Sensor Calibration

Sensor warm-up and calibration can affect MQ-135 readings.

FUTURE SCOPE

The system can be improved by:

- Adding an OLED/LCD display.
- Adding Wi-Fi connectivity.
- Adding LoRa communication.
- Storing sensor readings.
- Performing trend analysis.
- Developing food-specific calibration procedures.
- Adding additional sensors.

- Developing a mobile application.
- Developing a web dashboard.
- Providing historical alerts and monitoring.

These improvements are included in the original project's future-scope section.

CONCLUSION

The Food Spoilage Risk Indicator Using Embedded System demonstrates a simple embedded approach for monitoring environmental conditions associated with food spoilage risk.

The system combines temperature, humidity, and gas-response measurements using the DHT11 and MQ-135 sensors. The ESP32 processes the readings and compares them with selected threshold values.

When an alert condition occurs, the system provides an immediate audible warning using a buzzer. When the readings remain within the selected limits, the system indicates normal storage conditions.

The prototype is intended as an early-warning indicator rather than a direct food-safety certification system. Further calibration, food-specific experiments, additional sensing capabilities, and wireless monitoring can improve the system in future versions.

REFERENCES

1. Espressif Systems, ESP32 Series Documentation and Arduino-ESP32 Documentation.
2. DHT11 Temperature and Humidity Sensor Documentation/Datasheet.
3. MQ-135 Gas Sensor Module Documentation/Datasheet.
4. Arduino IDE Documentation and Arduino Programming Reference.

The references above are the references provided in the original project content.